

COMPETENCY-BASED LEARNING MATERIALS (CBLM)

AI BASIC PYTHON DEVELOPER

TECHNICAL AND VOCATIONAL CERTIFICATE I — LEVEL II

Core Competency: Apply Structured Computer Programming

Module Title: Apply Structured Computer Programming

Learning Outcome: LO3 – Execute Python Syntax

Nominal Duration: Theory 3.5 Hours / Practical 9 Hours

L.O 03: EXECUTE PYTHON SYNTAX

Upon completion of this learning outcome, students or trainees will have the following competencies:

1. Identify Python variables and apply appropriate variable naming rules.
2. Identify Python keywords and distinguish keywords from identifiers.
3. Identify common Python data types.
4. Identify Python numeric types.
5. Determine and specify Python variable types.
6. Apply correct Python indentation and whitespace.
7. Implement Python Boolean values and expressions.
8. Implement Python operators in simple programs.
9. Open and use Python IDLE or another Python development environment.
10. Create, save and execute Python program files.
11. Identify and correct basic syntax and runtime errors.
12. Apply basic debugging techniques and verify program output.

INFORMATION SHEET No. 03

Title: Understanding and Applying Python Syntax

Objective

Upon reading this Information Sheet, the trainee must be able to:

- Explain Python variables.
- Identify valid and invalid variable names.
- Identify Python keywords.
- Identify common Python data types.
- Distinguish Python numeric types.
- Determine variable types using `type()`.
- Specify and convert variable types.
- Explain indentation and whitespace.
- Implement Python Boolean values and expressions.
- Identify and apply common Python operators.
- Write simple Python statements using correct syntax.

1. Introduction to Python Syntax

Python syntax refers to the rules that determine how Python programs must be written.

Just as a spoken language has grammar rules, a programming language has syntax rules.

For example:

```
name = "Dara"  
print(name)
```

This code follows Python syntax.

Incorrect syntax may prevent Python from executing the program.

Example:

```
print("Hello"
```

The closing parenthesis is missing, so Python reports a syntax error.

2. Python Variables

A variable is a name used to store a value in computer memory.

Example:

```
name = "Dara"  
age = 20  
score = 85.5
```

In this example:

- name stores "Dara".
- age stores 20.
- score stores 85.5.

Python creates the variable when a value is assigned to it.

Unlike some programming languages, Python does not require the variable type to be declared before assigning the value.

Example:

```
number = 10
```

Python automatically recognizes 10 as an integer.

3. Variable Assignment

The equal sign = is the assignment operator.

Example:

```
x = 5
```

This means:

Store the value 5 in the variable x.

Another example:

```
student_name = "Sokha"  
student_age = 19
```

Variables can also store the result of an expression:

```
a = 5  
b = 10  
  
total = a + b
```

Here, total contains:

15

4. Python Variable Naming Rules

Python variable names must follow specific rules.

A variable name:

1. Must start with a letter or underscore _.

2. Cannot start with a number.
3. Can contain letters, numbers and underscores.
4. Cannot contain spaces.
5. Cannot be a Python keyword.
6. Is case-sensitive.

Valid variable names

```
name = "Dara"  
student_name = "Sokha"  
age2 = 20  
_total = 100  
studentScore = 90
```

Invalid variable names

```
2name  
student name  
student-name  
class
```

5. Case Sensitivity

Python is case-sensitive.

Example:

```
name = "Dara"  
Name = "Sokha"  
NAME = "Vanna"
```

Python considers these three different variables.

Therefore:

```
name  
Name  
NAME
```

are not the same identifier.

6. Python Keywords

Python keywords are reserved words that have special meanings in the Python language.

Examples include:

False	break	finally
None	class	for
True	continue	from
and	def	global
as	del	if
assert	elif	import
async	else	in
await	except	is

lambda
nonlocal
not
or

pass
raise
return
try

while
with
yield

Keywords should not be used as ordinary variable names.

Incorrect:

```
class = "Python"
```

Correct:

```
class_name = "Python"
```

7. Identifying Python Keywords

Python provides the built-in keyword module for checking keywords.

Example:

```
import keyword  
  
print(keyword.kwlist)
```

This displays the keywords recognized by the installed Python version.

A particular word can also be tested:

```
import keyword  
  
print(keyword.iskeyword("if"))  
print(keyword.iskeyword("student"))
```

Output:

```
True  
False
```

if is a Python keyword.

student is not a Python keyword.

8. Python Data Types

A data type describes the kind of value stored in a variable.

Common Python built-in data types include:

Category	Data Type	Example
Text	str	"Python"
Numeric	int	20
Numeric	float	85.5

Category	Data Type	Example
Numeric	complex	2 + 3j
Boolean	bool	True
Sequence	list	[1, 2, 3]
Sequence	tuple	(1, 2, 3)
Mapping	dict	{"name": "Dara"}
Set	set	{1, 2, 3}

Later learning outcomes will study lists, tuples, sets and dictionaries in greater detail.

For LO3, the main focus is:

- str
- int
- float
- complex
- bool

9. String Data Type

A string stores text.

Strings can be enclosed in single or double quotation marks.

Example:

```
name = "Dara"
course = 'Python'
```

Check the type:

```
(name))
```

Output:

```
<class 'str'>
```

10. Python Numeric Types

Python has three main built-in numeric types:

1. int
2. float
3. complex

10.1 Integer — int

An integer represents a whole number.

Examples:

```
age = 20
students = 25
year = 2026
temperature = -5
```

Check:

```
print(type(age))
```

Output:

```
<class 'int'>
```

10.2 Floating-Point Number — float

A float represents a number containing a decimal point.

Examples:

```
price = 25.50
score = 85.5
temperature = 36.7
```

Check:

```
print(type(score))
```

Output:

```
<class 'float'>
```

10.3 Complex Number — complex

A complex number contains a real and imaginary component.

Example:

```
number = 2 + 3j
```

Check:

```
print(type(number))
```

Output:

```
<class 'complex'>
```

11. Identifying a Variable Type

Python provides the `type()` function.

Syntax:

```
type(variable)
```

Example:

```
name = "Dara"
age = 20
```

```
score = 85.5
active = True

print(type(name))
print(type(age))
print(type(score))
print(type(active))
```

Output:

```
<class 'str'>
<class 'int'>
<class 'float'>
<class 'bool'>
```

12. Specifying and Converting Variable Types

Python provides functions for converting values from one compatible type to another.

Common conversion functions are:

```
int()
float()
str()
bool()
complex()
```

Convert string to integer

```
number = "10"

x = int(number)

print(x)
print(type(x))
```

Output:

```
10
<class 'int'>
```

Convert integer to float

```
number = 10

x = float(number)

print(x)
```

Output:

```
10.0
```

Convert number to string

```
age = 20
```

```
text_age = str(age)

print(type(text_age))
```

Output:

```
<class 'str'>
```

13. Python Indentation

Indentation means spaces placed at the beginning of a line of code.

Python uses indentation to define code blocks.

Example:

```
age = 20

if age >= 18:
    print("Adult")
```

The statement:

```
    print("Adult")
```

belongs to the if block and is therefore indented.

Incorrect:

```
if age >= 18:
print("Adult")
```

Python will report an indentation error.

14. Python Whitespace

Whitespace refers to spaces, tabs and blank lines in source code.

Appropriate whitespace makes programs easier to read.

Recommended:

```
a = 5
b = 10
total = a + b
```

Less readable:

```
a=5
b=10
total=a+b
```

A common Python convention is to use **four spaces per indentation level**.

Example:

```
score = 80

if score >= 50:
    print("Pass")
```

15. Python Boolean

The Boolean data type represents two values:

```
True
False
```

Example:

```
is_student = True
is_finished = False
```

Check:

```
print(type(is_student))
```

Output:

```
<class 'bool'>
```

Notice that Python uses:

```
True
False
```

with capital first letters.

16. Boolean Expressions

Boolean values are frequently produced through comparisons.

Example:

```
print(10 > 5)
```

Output:

```
True
```

Example:

```
print(10 < 5)
```

Output:

```
False
```

Variables can also be compared:

```
a = 5
b = 10

print(a < b)
```

Output:

True

17. Python Operators

Operators perform operations on variables and values.

Important categories include:

1. Arithmetic operators
2. Assignment operators
3. Comparison operators
4. Logical operators
5. Membership operators
6. Identity operators

18. Arithmetic Operators

Arithmetic operators perform mathematical operations.

Operator	Description	Example
+	Addition	5 + 2
-	Subtraction	5 - 2
*	Multiplication	5 * 2
/	Division	5 / 2
%	Modulus	5 % 2
**	Exponentiation	5 ** 2
//	Floor division	5 // 2

Example:

```
a = 5
b = 10

print("Addition:", a + b)
print("Subtraction:", b - a)
print("Multiplication:", a * b)
print("Division:", b / a)
```

Output:

```
Addition: 15
Subtraction: 5
Multiplication: 50
Division: 2.0
```

19. Assignment Operators

Assignment operators assign or update variable values.

Operator	Example	Equivalent
=	x = 5	Assign 5 to x
+=	x += 2	x = x + 2
-=	x -= 2	x = x - 2
*=	x *= 2	x = x * 2
/=	x /= 2	x = x / 2
%=	x %= 2	x = x % 2

Example:

```
x = 10
x += 5

print(x)
```

Output:

15

20. Comparison Operators

Comparison operators compare two values.

Operator	Description
==	Equal
!=	Not equal
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal

Example:

```
a = 5
b = 10

print(a == b)
print(a != b)
print(a < b)
print(a > b)
```

Output:

False
True
True
False

21. Logical Operators

Python provides three main logical operators:

Operator	Description
and	True when both conditions are true
or	True when at least one condition is true
not	Reverses a Boolean value

Example:

```
age = 20
score = 80

print(age >= 18 and score >= 50)
```

Output:

True

Example:

```
print(age < 18 or score >= 80)
```

Output:

True

Example:

```
is_active = True

print(not is_active)
```

Output:

False

22. Membership Operators

Membership operators determine whether a value exists in a sequence or collection.

They are:

```
in
not in
```

Example:

```
course = "Python"

print("P" in course)
print("Java" not in course)
```

Output:

```
True
True
```

23. Identity Operators

Python identity operators are:

```
is
is not
```

They determine whether two references refer to the same object.

Example:

```
value = None

print(value is None)
```

Output:

```
True
```

At this level, trainees are required mainly to recognize the purpose of identity operators.

24. Operator Precedence

When an expression contains multiple operators, Python follows operator precedence.

Example:

```
result = 5 + 2 * 3

print(result)
```

Output:

```
11
```

Multiplication is evaluated before addition.

Parentheses can change the order:

```
result = (5 + 2) * 3

print(result)
```

Output:

```
21
```

Using parentheses can also make expressions easier to understand.

25. Combining Variables, Data Types, Boolean and Operators

Example:

Python syntax demonstration

```
student_name = "Dara"
age = 20
score = 85.5

a = 5
b = 10

total = a + b
is_adult = age >= 18

print("Student:", student_name)
print("Age:", age)
print("Score:", score)

print("Total:", total)
print("Adult:", is_adult)
```

Output:

```
Student: Dara
Age: 20
Score: 85.5
Total: 15
Adult: True
```

This simple program demonstrates:

- Variables
- Data types
- Numeric values
- Arithmetic operator
- Comparison operator
- Boolean value
- Correct Python syntax

26. Common Python Syntax Errors

Invalid Variable Name

Incorrect:

```
2name = "Dara"
```

Correct:

```
name2 = "Dara"
```

Using a Keyword as Variable Name

Incorrect:

```
class = "Python"
```

Correct:

```
class_name = "Python"
```

Incorrect Indentation

Incorrect:

```
age = 20

if age >= 18:
print("Adult")
```

Correct:

```
age = 20

if age >= 18:
    print("Adult")
```

Missing Quotation Mark

Incorrect:

```
name = "Dara
```

Correct:

```
name = "Dara"
```

Incorrect Variable Reference

Example:

```
student_name = "Dara"

print(studentName)
```

Python reports an error because studentName has not been defined.

Correct:

```
print(student_name)
```

27. Basic Debugging

Debugging is the process of finding and correcting errors in a program.

A basic debugging sequence is:

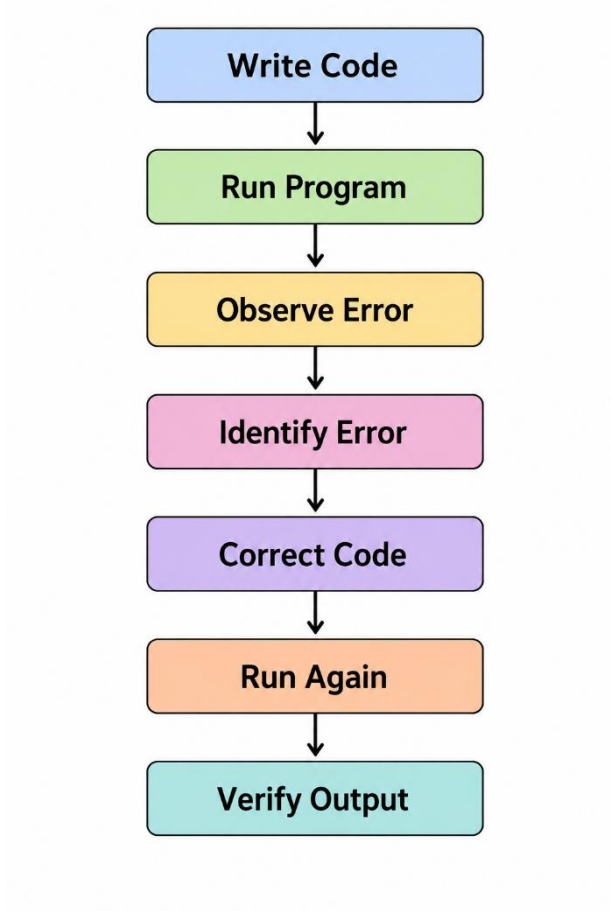


Figure 1: Basic Debugging

When an error occurs:

1. Read the error message.
2. Check the reported line.
3. Check variable names.
4. Check quotation marks.
5. Check parentheses.
6. Check indentation.
7. Check the operator being used.
8. Correct the code.
9. Run the program again.
10. Verify the output.

SELF-CHECK No. 3.1

A. Multiple Choice

1. What is a Python variable?

- A. A Python error
- B. A name used to store a value
- C. A computer device
- D. A Python file

2. Which is a valid Python variable name?

- A. 2student
- B. student name
- C. student_name
- D. class

3. Which is a Python keyword?

- A. student
- B. price
- C. if
- D. score

4. Which data type represents a whole number?

- A. str
- B. float
- C. int
- D. bool

5. Which data type represents 25.50?

- A. int
- B. float
- C. str
- D. complex

6. Which is a Python numeric type?

- A. int
- B. str
- C. list
- D. dict

7. Which function identifies a variable's type?

- A. data()
- B. check()
- C. type()
- D. variable()

8. Which function converts "10" to an integer?

- A. str()
- B. int()
- C. float()
- D. bool()

9. Which are Python Boolean values?

- A. Yes and No
- B. 1 and 0
- C. True and False
- D. On and Off

10. Which operator performs multiplication?

- A. +
- B. -
- C. *
- D. /

11. Which operator checks equality?

- A. =
- B. ==
- C. !=
- D. >=

12. Which logical operator requires both conditions to be true?

- A. and
- B. or
- C. not
- D. is

ANSWER KEY No. 3.1

A. Multiple Choice

No.	Answer
1	B
2	C
3	C
4	C
5	B
6	A
7	C
8	B
9	C
10	C
11	B
12	A

OPERATION SHEET No. 3.1

Title: Create Variables, Identify Data Types and Apply Python Operators

Performance Objective

Given a computer with Python installed, Python IDLE or another approved development environment, the trainee must create variables, identify their data types, apply Boolean expressions and operators, save the program and execute it successfully.

Description

This operation demonstrates how to create Python variables and use the basic syntax covered in LO3.

Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Desktop Computer/Laptop	Unit	1
2	Keyboard and Mouse	Set	1
3	Python 3.x	Copy	1
4	Python IDLE/IDE	Software	1

Supplier

Training institution / authorized software provider.

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Open Python IDLE/IDE	Development environment opens successfully
2	Create a new Python file	New source file is created
3	Create variables	Valid variable names are used
4	Assign different values	Appropriate values are stored
5	Use type()	Correct data types are identified
6	Apply type conversion	Compatible values are converted correctly
7	Create Boolean values	True and False are used correctly
8	Apply arithmetic operators	Correct mathematical results are produced
9	Apply comparison operators	Correct Boolean results are produced
10	Apply logical operators	Correct logical results are produced
11	Apply indentation	Code block is correctly indented
12	Save program	.py extension is used
13	Execute program	Program executes successfully
14	Verify results	Actual results match expected results

Operation

Create a new file and enter:

```
student_name = "Dara"  
age = 20  
average = 85.5
```

```

is_student = True
print(student_name)
print(age)
print(average)
print(is_student)
Identify the data types:
print(type(student_name))
print(type(age))
print(type(average))
print(type(is_student))

```

Apply operators:

```

a = 5
b = 10
print("Addition:", a + b)
print("Subtraction:", b - a)
print("Multiplication:", a * b)
print("Division:", b / a)

```

Apply comparison operators:

```

print(a < b)
print(a > b)
print(a == b)
print(a != b)

```

Apply logical operators:

```

print(a < b and b == 10)
print(a > b or b == 10)
print(not a > b)

```

Apply indentation:

```

if a < b:
    print("A is less than B")

```

Save as:

```
python_syntax.py
```

Run the program and verify the results.

Assessment Method

- Direct observation
- Practical demonstration
- Source-code inspection
- Program execution
- Output verification

TASK SHEET No. 3.1

Title: Identify Python Variables, Keywords and Data Types

Performance Task

Create Python variables and identify valid variable names, Python keywords and data types.

Performance Objective

Given Python IDLE/IDE and a worksheet, the trainee must correctly create variables, identify keywords and determine variable data types.

Description

The trainee will create variables representing student information and use `type()` to identify their data types.

Picture/Overview

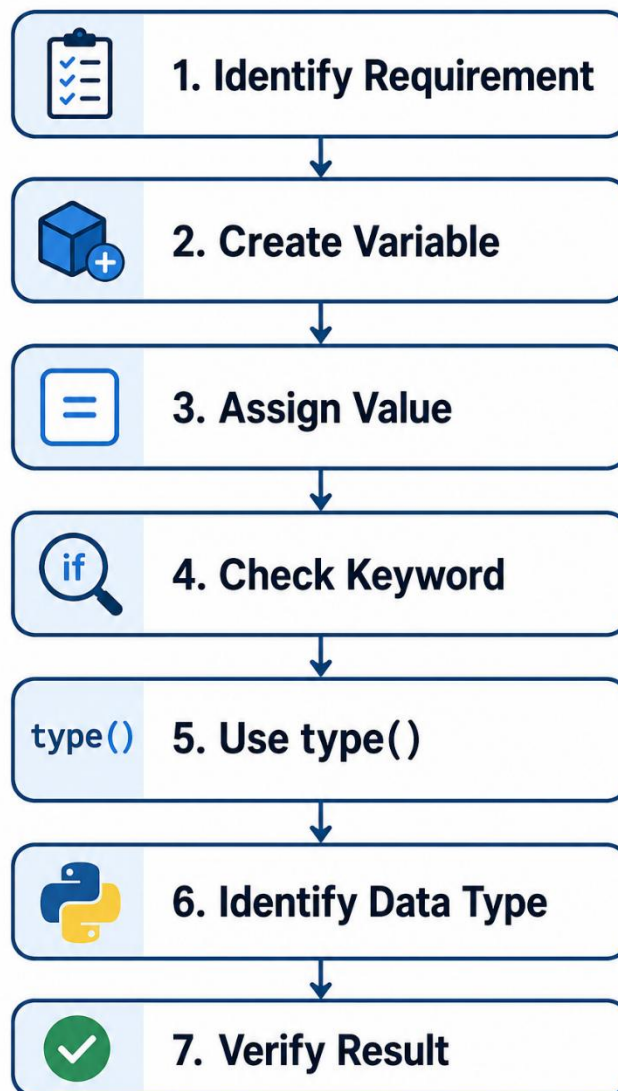


Figure 2: Identify Python Variables

Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Computer/Laptop	Unit	1
2	Python 3.x	Copy	1
3	Python IDLE/IDE	Software	1
4	Programming Worksheet	Copy	1
5	Python Reference Book	Copy	1

Supplier

Training institution.

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Open Python IDE	IDE opens successfully
2	Create new file	Python source file is created
3	Create variables	Valid naming rules are followed
4	Identify keywords	Reserved words are correctly recognized
5	Assign values	Appropriate values are assigned
6	Apply type()	Data types are correctly identified
7	Save file	.py extension is used
8	Execute file	Program runs successfully
9	Verify output	Types match expected types

Practical Exercise

Create:

```
student_name = "Dara"  
age = 20  
average = 85.5  
complex_number = 2 + 3j  
is_student = True
```

Display each value and type.

Complete:

Variable	Value	Expected Type	Actual Type
student_name	"Dara"	_____	_____
age	20	_____	_____
average	85.5	_____	_____
complex_number	2 + 3j	_____	_____
is_student	True	_____	_____

Identify whether the following are Python keywords:

Word	Keyword? Yes/No
if	
student	
class	
score	
while	
total	

Assessment Method

- Direct observation
- Worksheet review
- Oral questioning
- Source-code inspection

TASK SHEET No. 3.2

Title: Specify Python Variable Types

Performance Task

Identify and convert Python variable types.

Performance Objective

Given different Python values, the trainee must use `type()` to identify their data types and use appropriate conversion functions to specify compatible variable types.

Description

The trainee will practice using:

```
type()
int()
float()
str()
bool()
```

Picture/Overview

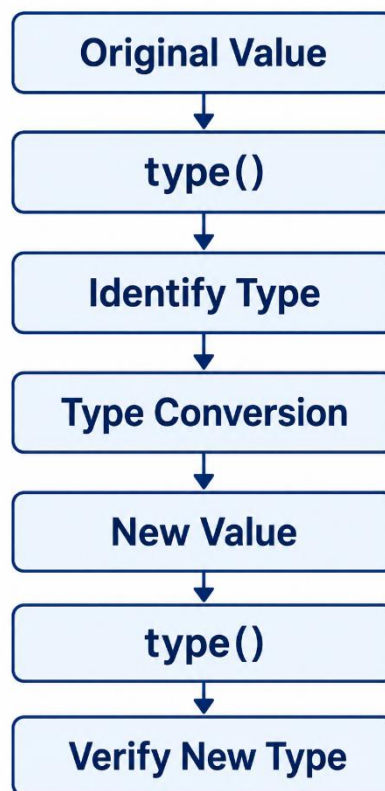


Figure 3: Variable Types

Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Computer/Laptop	Unit	1
2	Python 3.x	Copy	1

No.	Description	Unit	Qty
3	Python IDLE/IDE	Software	1
4	Worksheet	Copy	1

Supplier

Training institution.

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	1. Create variables	Values are correctly assigned
2	Use type()	Original types are identified
3	Apply int()	Compatible value is converted to integer
4	Apply float()	Compatible value is converted to float
5	Apply str()	Value is converted to string
6	Check converted types	Correct types are displayed
7	Save and execute	Program runs correctly

Practical Exercise

Enter:

```
number_text = "25"
age = 20
price = 50
```

```
number = int(number_text)
age_float = float(age)
price_text = str(price)

print(number, type(number))
print(age_float, type(age_float))
print(price_text, type(price_text))
```

Complete:

Original	Original Type	Conversion	New Type
"25"	_____	int()	_____
20	_____	float()	_____
50	_____	str()	_____

Assessment Method

- Practical demonstration
- Direct observation
- Worksheet review
- Output verification

TASK SHEET No. 3.3

Title: Implement Python Boolean and Operators

Performance Task

Use Python variables, Boolean expressions and operators to perform calculations and comparisons.

Performance Objective

Given values and programming requirements, the trainee must correctly implement arithmetic, assignment, comparison and logical operators.

Description

Use:

```
a = 5  
b = 10
```

Perform arithmetic and Boolean operations.

Picture/Overview

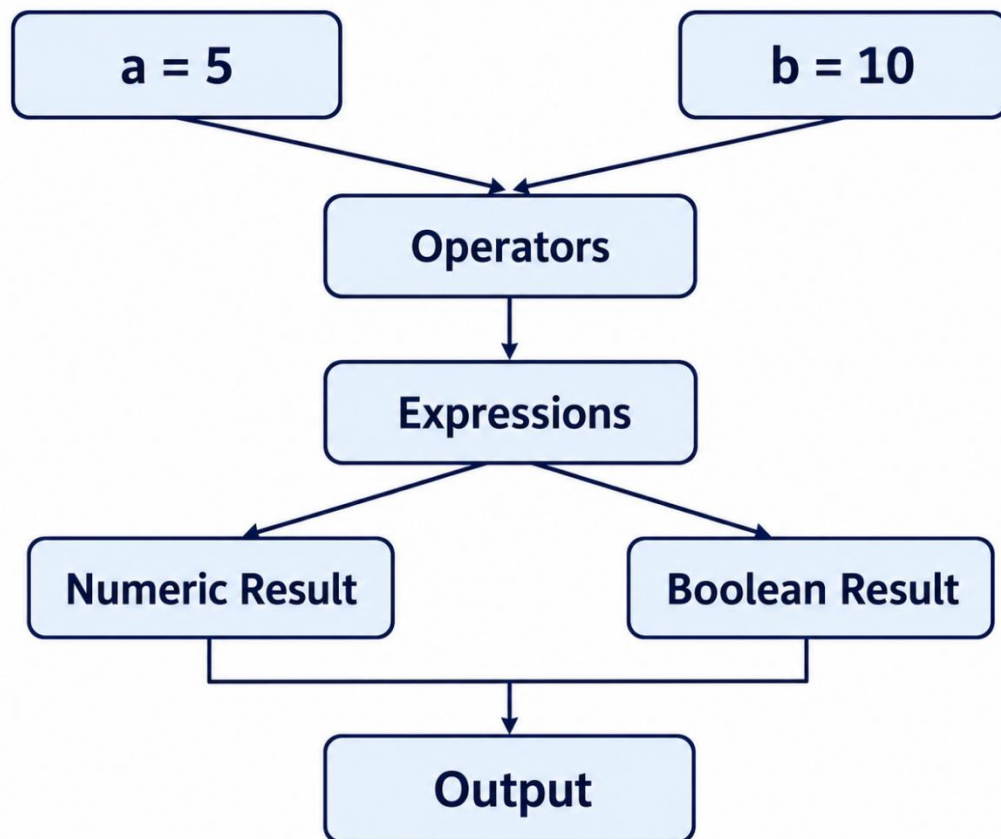


Figure 4: Use Python variables

Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Computer/Laptop	Unit	1
2	Python 3.x	Copy	1
3	Python IDE	Software	1
4	Worksheet	Copy	1
5	Python Reference Book	Copy	1

Supplier

Training institution.

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Assign a = 5	Correct value is assigned
2	Assign b = 10	Correct value is assigned
3	Apply arithmetic operators	Correct results are produced
4	Apply comparison operators	Correct Boolean values are produced
5	Apply logical operators	Correct Boolean values are produced
6	Display results	Required results are shown
7	Save program	.py extension is used
8	Execute program	Program runs successfully
9	Verify results	Output is correct

Practical Exercise

```
a = 5
b = 10

print("Addition:", a + b)
print("Subtraction:", b - a)
print("Multiplication:", a * b)
print("Division:", b / a)
print("Modulus:", b % a)
print("Exponent:", a ** 2)

print("A < B:", a < b)
print("A > B:", a > b)
print("A == B:", a == b)
print("A != B:", a != b)

print("Logical AND:", a < b and b == 10)
print("Logical OR:", a > b or b == 10)
print("Logical NOT:", not a > b)
```

Complete:

Expression	Expected Result	Actual Result
a + b	15	
b - a	5	
a * b	50	
b / a	2.0	
a < b	True	
a > b	False	
a == b	False	
a != b	True	

Assessment Method

- Direct observation
- Practical demonstration
- Program execution
- Output verification

TASK SHEET No. 3.4

Title: Apply Python Indentation and Correct Syntax Errors

Performance Task

Identify and correct indentation and basic syntax errors in Python programs.

Performance Objective

Given faulty Python code, the trainee must identify the errors, correct the syntax and indentation, execute the corrected program and verify its output.

Description

The trainee will debug simple Python code containing invalid variable names, incorrect indentation and basic syntax errors.

Picture/Overview

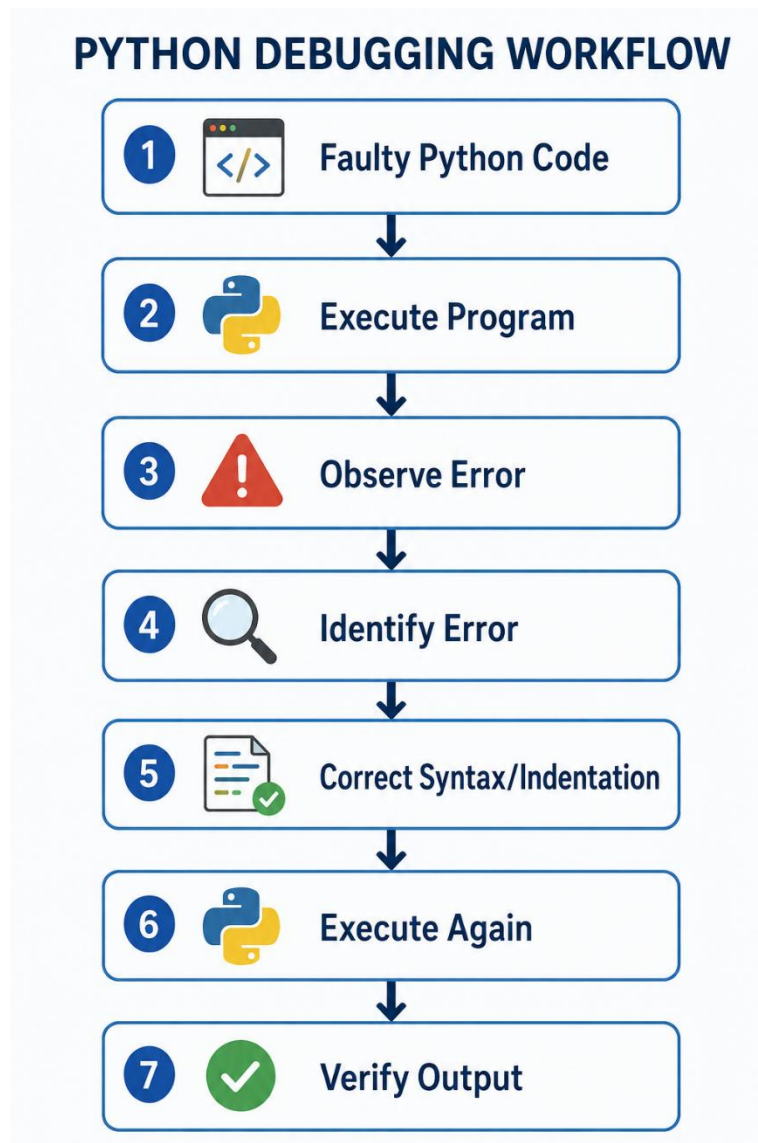


Figure 5: Python Basic Debugging Error

Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Computer/Laptop	Unit	1
2	Python 3.x	Copy	1
3	Python IDLE/IDE	Software	1
4	Error Exercise Files	Set	1
	Worksheet	Copy	1

Supplier

Training institution.

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Open faulty program	File opens successfully
2	Execute program	Error is observed
3	Read error message	Error information is identified
4	Locate error	Correct line is identified
5	Identify error type	Basic error is correctly recognized
6	Correct syntax	Python syntax is corrected
7	Correct indentation	Code block is properly indented
8	Execute again	Corrected program runs
9	Verify output	Expected result is produced

Exercise 1

Incorrect:

```
2name = "Dara"  
print(2name)
```

Correct the program.

Exercise 2

Incorrect:

```
age = 20  
if age >= 18:  
    print("Adult")
```

Correct the program.

Exercise 3

Incorrect:

```
name = "Dara"  
print(name)
```

Correct the program.

Exercise 4

Incorrect:

```
a = 5
b = 10
total = a + c
print(total)
```

Identify the error and correct the program so the result is:

15

Assessment Method

- Direct observation
- Debugging exercise
- Source-code inspection
- Program execution
- Output verification

JOB SHEET No. 3.5

Title: Develop a Python Syntax Demonstration Program

Performance Task

Develop a complete Python program demonstrating variables, keywords awareness, data types, numeric types, variable type specification, indentation, Boolean values and operators.

Performance Objective

Given a computer, Python IDLE or another approved Python development environment, programming requirements and reference materials, the trainee must analyze the requirements, create a Python program using correct syntax, save the program with a .py extension, execute and debug the program, and verify its output.

Description

Develop a **Student Programming Profile and Calculation Program**.

The program must demonstrate:

1. Python variables.
2. Valid variable naming.
3. String data type.
4. Integer data type.
5. Floating-point data type.
6. Complex numeric type.
7. Boolean data type.
8. `type()` function.
9. Type conversion.
10. Arithmetic operators.
11. Comparison operators.
12. Logical operators.
13. Correct indentation.
14. Correct program execution.
15. Output verification.

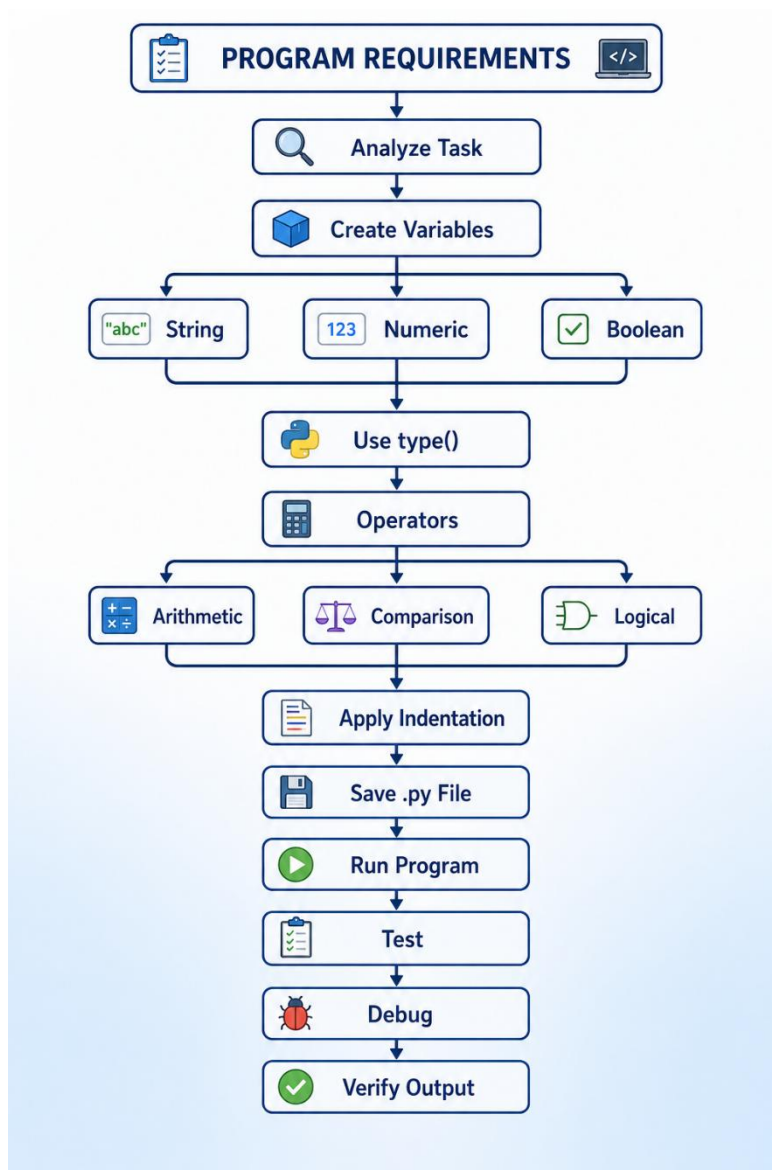


Figure 6: Develop a Python Syntax Demonstration Program

Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Desktop Computer/Laptop	Unit	1
2	Keyboard and Mouse	Set	1
3	Python 3.x	Copy	1
4	Python IDLE/IDE	Software	1
5	Command Prompt/Terminal	Software	1
6	Python Programming Reference Book	Copy	1
7	Programming Worksheet	Copy	1
8	USB Flash Drive	Unit	1

Supplier

Training institution / authorized software provider.

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Analyze program requirements	Required input, process and output are identified
2	Open Python IDE	Development environment opens successfully
3	Create Python file	New source file is created
4	Create variables	Valid Python identifiers are used
5	Avoid reserved keywords	Python keywords are not used incorrectly
6	Assign string value	Correct str value is created
7	Assign integer value	Correct int value is created
8	Assign float value	Correct float value is created
9	Assign complex value	Correct complex value is created
10	Assign Boolean value	Correct bool value is created
11	Use type()	Variable types are correctly identified
12	Apply type conversion	Compatible type conversion is completed
13	Apply arithmetic operators	Correct numeric results are produced
14	Apply comparison operators	Correct Boolean results are produced
15	Apply logical operators	Correct logical results are produced
16	Apply indentation	Python code block is correctly indented
17	Save source file	.py extension is used
18	Execute program	Program runs successfully
19	Identify errors	Basic syntax/runtime errors are identified
20	Correct errors	Appropriate debugging is performed
21	Execute corrected program	Program runs without identified errors
22	Verify output	Program output matches requirements

Program Requirements

Create the file:

```
python_syntax_demo.py
```

Create variables containing:

```
Student Name
```

```
Age
```

```
Average Score
```

```
Complex Number
```

```
Student Status
```

Use values similar to:

```
student_name = "Dara"
```

```
age = 20
```

```
average_score = 85.5
```

```
complex_number = 2 + 3j
is_student = True
```

Display the type of each variable.

Create:

```
a = 5
b = 10
```

Calculate:

```
addition = a + b
subtraction = b - a
multiplication = a * b
division = b / a
```

Create Boolean comparisons using:

```
<
>
==
!=
```

Use at least:

```
and
or
not
```

Use at least one correctly indented Python code block.

Example Program Structure

```
# Python Syntax Demonstration Program
```

```
student_name = "Dara"
age = 20
average_score = 85.5
complex_number = 2 + 3j
is_student = True

print("Student Name:", student_name)
print("Age:", age)
print("Average Score:", average_score)
print("Complex Number:", complex_number)
print("Student Status:", is_student)

print("\nDATA TYPES")
print(type(student_name))
print(type(age))
print(type(average_score))
print(type(complex_number))
print(type(is_student))
```

```
a = 5
```

```

b = 10

print("\nARITHMETIC OPERATORS")
print("A + B =", a + b)
print("B - A =", b - a)
print("A * B =", a * b)
print("B / A =", b / a)

print("\nCOMPARISON OPERATORS")
print("A < B:", a < b)
print("A > B:", a > b)
print("A == B:", a == b)
print("A != B:", a != b)

print("\nLOGICAL OPERATORS")
print(a < b and b == 10)
print(a > b or b == 10)
print(not a > b)

print("\nINDENTATION")
if a < b:
    print("A is less than B")

```

Expected Output

```

Student Name: Dara
Age: 20
Average Score: 85.5
Complex Number: (2+3j)
Student Status: True

```

DATA TYPES

```

<class 'str'>
<class 'int'>
<class 'float'>
<class 'complex'>
<class 'bool'>

```

ARITHMETIC OPERATORS

```

A + B = 15
B - A = 5
A * B = 50
B / A = 2.0

```

COMPARISON OPERATORS

```

A < B: True
A > B: False

```

```
A == B: False
A != B: True
LOGICAL OPERATORS
True
True
True
INDENTATION
A is less than B
```

WORKSHEET No. 3.1

Title: Python Syntax Practice and Verification Worksheet

Trainee Name: _____

Date: _____

Part A — Variable Analysis

Complete the table.

Variable	Value	Data Type
student_name	"Dara"	_____
age	20	_____
average_score	85.5	_____
complex_number	2 + 3j	_____
is_student	True	_____

Part B — Variable Naming

Write **Valid** or **Invalid**.

Variable Name	Answer
student_name	_____
2student	_____
student2	_____
student name	_____
_score	_____
class	_____

Part C — Operator Results

Given:

a = 5

b = 10

Complete:

Expression	Result
a + b	_____
b - a	_____
a * b	_____
b / a	_____
b % a	_____
a ** 2	_____
a < b	_____
a > b	_____
a == b	_____
a != b	_____

Part D — Boolean

Given:

```
a = 5
b = 10
```

Determine:

Expression	True/False
a < b	_____
a > b	_____
a < b and b == 10	_____
a > b or b == 10	_____
not a > b	_____

Part E — Debugging

Correct:

```
2name = "Dara"
```

Correct code:

Correct:

```
class = "Python"
```

Correct code:

Correct:

```
age = 20
```

```
if age >= 18:
    print("Adult")
```

Correct code:

PERFORMANCE CHECKLIST No. 3.1

Title: Execute Python Syntax

Name: _____

Date: _____

Did you use all these performance criteria during your task?

No.	Criteria	Yes	No
1	Identified Python variables correctly	☐	☐
2	Applied valid Python variable naming rules	☐	☐
3	Assigned values to Python variables correctly	☐	☐
4	Identified Python keywords correctly	☐	☐
5	Distinguished keywords from ordinary identifiers	☐	☐
6	Identified common Python data types	☐	☐
7	Identified str data type	☐	☐
8	Identified int numeric type	☐	☐
9	Identified float numeric type	☐	☐
10	Identified complex numeric type	☐	☐
11	Identified bool data type	☐	☐
12	Used type() to identify variable types	☐	☐
13	Applied appropriate type conversion	☐	☐
14	Identified indentation correctly	☐	☐
15	Applied correct indentation to code blocks	☐	☐
16	Applied appropriate whitespace	☐	☐
17	Implemented True and False correctly	☐	☐
18	Created Boolean expressions correctly	☐	☐
19	Applied arithmetic operators correctly	☐	☐
20	Applied assignment operators correctly	☐	☐
21	Applied comparison operators correctly	☐	☐
22	Applied logical operators correctly	☐	☐
23	Analyzed simple program requirements	☐	☐
24	Identified required inputs, processes and outputs	☐	☐
25	Opened and used Python IDLE/IDE correctly	☐	☐
26	Wrote Python code using correct syntax	☐	☐
27	Created and saved the program with .py extension	☐	☐
28	Executed the Python program successfully	☐	☐
29	Identified basic syntax errors	☐	☐

No.	Criteria	Yes	No
30	Identified basic runtime errors	☐	☐
31	Corrected identified errors	☐	☐
32	Applied basic debugging techniques	☐	☐
33	Re-executed the corrected program	☐	☐
34	Verified the final program output	☐	☐

Comment:

Overall Assessment

Competent: ☐ Yes ☐ No

Needs Further Practice: ☐ Yes ☐ No

Name and Signature of Trainer:

Name:

Signature:

Date:
